

Programación Orientada a Objetos
Trabajo Práctico Integrador
UNQ-Shop

Integrantes:

Santiago Catalogna

Mail: catalognasantiago@gmail.com

Santiago Vazquez

Mail: santivaz232@gmail.com

Fernández Ramiro

Mail: ramyfernandez3@gmail.com

Repositorio: <https://github.com/Masque87/unqui-poo2-catalogna-integrador>

Decisiones de diseño y Detalles de implementación

A continuación, se detallarán patrones usados, roles de cada clase, decisiones de diseño y detalles de implementación dignos de mencionar, divididos por punto de actividad:

2.1) Catálogo de productos

Patron utilizado: Composite

Razones: Ambos paquete y producto comparten un comportamiento que me interesa tratarlos por igual, luego el paquete tiene productos, ambos además comparten un comportamiento en común, definido en la clase padre ItemCatalogo.

Roles de cada clase:

- ItemCatalogo: Component
- Producto: Leaf
- Paquete: Composite

Detalles de implementación:

Para la implementación de atributos dinámicos en el producto se decidió usar un tipo de dato llamado map, basándome en que es una posible solución de mi gusto al tener el nombre de los atributos separados en claves, y sus atributos en values, facilitando las verificaciones con un mayor orden.

Las verificaciones solo se hacen en atributos claves, otros como la descripción se decidió no hacerlos obligatorios, para permitir mayor flexibilidad al crear un producto.

ItemCatalogo es abstracta, Se decide que sea una clase abstracta ya que no hay necesidad de instanciar un ItemCatalogo en el contexto de enunciado y necesidades del ejercicio, luego no se implementa add, remove y getChild ya que entraría en conflicto con la leaf producto.

2.2) Ciclo de vida del pedido

Patrón utilizado: State / Strategy

Razones: Hay operaciones que son válidas en ciertos estados del pedido e inválidas en otros. Implementar un patrón State nos permite separar esos comportamientos de una manera eficaz. Sin embargo, gran parte de las operaciones se repite entre los estados, variando sólo su implementación. Es por este motivo que creemos que también tiene elementos de un patrón Strategy.

Roles de cada clase:

- Pedido: Context.
- Estado(interfaz): State.
- Borrador, Confirmado, Enviado, etc (las subclases de EstadoPedido): Concrete States.

Detalles de implementación:

- Cada pedido inicia en estado de Borrador.
- El pedido se comunica con su estado a través de una interfaz y cada estado concreto almacena el pedido al que pertenece.
 - Cada estado concreto se comunica directamente con su pedido.
- Se decidió crear una superclase abstracta EstadoPedido con la función principal de albergar el código en común entre los distintos estados del pedido, evitando así tener que definir muchos métodos vacíos.
- Se usan timestamps en los estados Enviado y Entregado para informar la fecha en la que se completó cada fase.
- Cada vez que el pedido avanza al siguiente estado, se reemplaza la instancia del estado que tenía por una instancia del nuevo estado. Luego de eso, se realizan automáticamente las operaciones propias del estado.

2.3) Medios de Envío

Patrón utilizado: Strategy

Razones: Todos los medios de envío hacen lo mismo: calculan el costo del envío y el tiempo de entrega. Lo que difiere es cómo lo hace cada uno.

Roles de cada clase:

Pedido: Context.

MetodoDeEnvio: Strategy.

RetiroSucursal, EnvioExpress, EnvioEstandar: Concrete Strategies.

Detalles de implementacion:

- A cada pedido se le asigna un metodo de envio en el momento de ser instanciado.
- Debido a que en EnvioExpress y EnvioEstandar no hay realmente una manera definida de calcular el tiempo de envío de forma precisa, y a que en RetiroSucursal el tiempo siempre es el mismo, se hardcodea un string en el mensaje que lo consulta. Podría ser refactorizado a futuro.

2.4) Métodos de pago

Patron utilizado: Template Method

Razones: Comparten mismo algoritmo al procesar un pago los distintos medios de pago.

Roles De Cada Clase:

- MedioDePago: AbstractClass
- TarjetaDeCredito: ConcreteClass
- TransferenciaBancaria: ConcreteClass
- BilleteraVirtual: ConcreteClass

(PagoException, TarjetaDeCreditoAPI, TransferenciaBancariaAPI, BilleteraVirtualAPI no influncian en el patron)

Detalles de implementación:

Ante el aumento de validaciones en este punto se creó PagoException para tener unas validaciones más personalizadas.

Se tomó la decisión de no guardar los medios de pago utilizados, debido a que en el contexto de ecommerce sería raro mantener la tarjeta de credito de un cliente en la bdd, por lo tanto cada medio de pago tiene un monto fijo y guarda solo un recibo y cada paso del algoritmo no pide un monto de este modo, ya que lo tiene guardado en la clase.

El generarCodigoTransaccion utiliza System.currentTimeMillis() ya que pareció una buena idea para generar codigos únicos y sería util en este contexto.

2.5) Notificaciones Del Pedido

Patron utilizado: Observer

Razones: Se determinó utilizar el patrón Observer porque cada subsistema está pendiente a los cambios de estado del pedido y reacciona sólo al estado que le corresponde.

Roles de cada clase:

- Pedido: Publisher
- Notificable: Subscriber
- Fidelizacion, GeneradorFactura, NotificadorEmail: Concrete subscribers

Detalles de implementacion:

El pedido tiene una lista de notificables, con la cual se comunica por medio de la interfaz Notificable cada vez que cambia de estado, informando el estado anterior y el estado nuevo.

Cada notificable concreto realiza sus operaciones enviándole un mensaje al estado nuevo y valiéndose del double dispatch para que el mensaje enviado surta efecto.

2.6 Búsqueda en el catálogo

Patron utilizado: Strategy y Composite

Razones: Necesidad de poder interactuar con todos los criterios de búsqueda de la misma forma sin tener que prejuzgar por cada tipo, y strategy por la necesidad de variar en comportamientos de algoritmos pero que comparten una misma naturaleza reduciendo casos if.

Roles de cada clase:

Catalogo: Context

CriterioBusqueda: Strategy y Component

CriterioCompuesto: Component

CriterioPorNombre: ConcreteStrategy y leaf

CriterioPorPrecioMaximo: ConcreteStrategy y leaf

CriterioPorCategoria: ConcreteStrategy y leaf

CriterioPorDisponibilidad: ConcreteStrategy y leaf

CriterioAND: ConcreteStrategy y Composite

CriterioOR: ConcreteStrategy y Composite

CriterioNOT: ConcreteStrategy

Detalles de implementación:

CriterioCompuesto se creó debido a que AND y OR comparten patrones de comportamientos comunes, pudiendo entonces ahorrar el repetir código, NOT no comparte coincidencias entre estos dos por lo tanto no hereda de CriterioCompuesto. A parte de esto no se encontró mucho más código repetido entre estrategias para tener una clase padre en común, por lo que en cambio cada uno implementa una interfaz.

2.8) Generación de reportes

Patrón utilizado: Visitor

Razones: Todos los reportes tienen el mismo comportamiento, son Visitantes concretos los cuales visitan las subclases de ItemCatalogo para saber como deben ser impresos en distintos formatos de texto.

Roles de cada clase:

- Reporte: Visitor
- ReporteTextoPlano, ReporteCSV y ReporteHTML : ConcreteVisitor
- ItemCatalogo: Element

Detalles de la implementación: Se optó por trabajar los reportes considerando que su periodo es desde el conocimiento de los tiempos hasta el día y momento de generado el reporte, es decir, es un reporte histórico de productos más vendidos.

También, se analizó la posibilidad de contabilizar solo aquellos Pedidos que se encuentren en estado “Entregado” ya que es un estado terminal.

